

Parallel & Distributed Computing



PROCESSES AND MESSAGE PASSING IN DISTRIBUTED SYSTEM



Dr. Muzamil Ahmed (Assistant Professor)
Namal University Mianwali

Lesson from Quran

2

قُلْ لَا يَعْلَمُ مَنْ فِي السَّمَاوَاتِ
وَالْأَرْضِ الْغَيْبَ إِلَّا اللَّهُ

No one in the heavens and the earth
knows the unseen except Allah

Quran 26:65

Lecture Outline

3

- Message Passing in Distributed System
- Importance of Message Passing
- Types of Message Passing
- Message-passing Programming
- Asynchronous vs Loosely synchronous Paradigm
- Send and Receive Operations
- Blocking Buffered vs Non-Buffered Send/Receive



Message Passing in Distributed System

- Message passing in distributed systems refers to the communication medium used by nodes (computers or processes) to communicate information and coordinate their actions.
- It involves transferring and entering messages between nodes to achieve various goals such as coordination, synchronization, and data sharing.

What is Message Passing in Distributed Systems?

- The method by which entities or processes in a distributed system communicate and exchange data is known as message passing.
- It enables several components, which could be operating on different computers or nodes connected by a network, to plan their actions, exchange data, and work together to accomplish shared objectives.
 - ▣ Models like synchronous and asynchronous message passing offer different synchronization and communication semantics to suit system requirements.
 - ▣ Synchronous message passing ensures sender and receiver synchronization, while asynchronous message passing allows concurrent execution and non-blocking communication.

Importance of Message Passing

- ❑ In distributed systems, where multiple independent components work together to perform tasks, message passing is crucial for inter-process communication (IPC).
- ❑ It enables applications to distribute workloads, share resources, synchronize actions, and handle concurrent activities across different nodes efficiently.

Types of Message Passing in Distributed Systems

- Message passing describes the method by which nodes or processes interact and share information in distributed systems.
- Message passing can be divided into two main categories according to the sender and receiver's timing and synchronization:
 - ▣ Synchronous Message Passing
 - ▣ Asynchronous Message Passing
 - ▣ Unicast Messaging
 - ▣ Multicast Messaging
 - ▣ Broadcast Messaging

Synchronous Message Passing

- Synchronous message passing involves a tightly coordinated interaction between the sender and receiver. The key characteristics include:
 - ▣ **Timing Coordination:** Before proceeding with execution, the sender waits for the recipient to confirm receipt of the message or finish processing it.
 - ▣ **Request-Response Pattern:** often use a request-response paradigm in which the sender sends a message requesting something and then waits for the recipient to react.
- **Advantages:**
 - ▣ Ensures precise synchronization between communicating entities.
 - ▣ Simplifies error handling as the sender knows when the message has been successfully received or processed.
- **Disadvantages:**
 - ▣ May introduce latency if the receiver is busy or unavailable.
 - ▣ Synchronous blocking can reduce overall system throughput if many processes are waiting for responses.

Asynchronous Message Passing

- Asynchronous message passing allows processes to operate independently of each other in terms of timing. Key features include:
 - ▣ **Decoupled Timing:** The sender does not wait for an immediate response from the receiver after sending a message. It continues its execution without blocking.
 - ▣ **Event-Driven Model:** Communication is often event-driven, where processes respond to messages or events as they occur asynchronously.
- **Advantages:**
 - ▣ Enhances system responsiveness and throughput by allowing processes to execute concurrently.
 - ▣ Allows for interactions that are loosely connected, allowing processes to process messages at their own speed.
- **Disadvantages:**
 - ▣ Requires additional mechanisms (like callbacks or event handlers) to manage responses or coordinate actions.
 - ▣ Handling out-of-order messages or ensuring message delivery reliability can be more complex compared to synchronous communication.

Unicast Messaging

- ❑ Unicast messaging is a one-to-one communication where a message is sent from a single sender to a specific receiver. The key characteristics include:
 - ▣ **Direct Communication:** The message is targeted at a single, specific node or endpoint.
 - ▣ **Efficiency for Point-to-Point:** Since only one recipient receives the message, resources are efficiently used for direct, point-to-point communication.
- ❑ **Advantages:**
 - ▣ Optimized for targeted communication, as the message is only sent to the intended recipient.
 - ▣ Minimizes network load compared to group messaging, as it doesn't broadcast to unnecessary nodes.
- ❑ **Disadvantages:**
 - ▣ Not scalable for group communications; sending multiple unicast messages can strain the system in larger networks.
 - ▣ Can increase the complexity of managing multiple unicast connections in large-scale applications.

Multicast Messaging

- Multicast messaging enables one-to-many communication, where a message is sent from one sender to a specific group of receivers. The key characteristics include:
 - ▣ **Group-Based Communication:** Messages are delivered to a subset of nodes that have joined the multicast group.
 - ▣ **Efficient for Groups:** Saves bandwidth by sending the message once to all nodes in the group instead of individually.
- **Advantages:**
 - ▣ Reduces network traffic by sending a single message to multiple recipients, making it ideal for content distribution or group updates.
 - ▣ Scales efficiently for applications where data needs to reach specific groups, like video conferencing or online gaming.
- **Disadvantages:**
 - ▣ Complex to implement as nodes need mechanisms to manage group memberships and handle node join/leave requests.
 - ▣ Not all network infrastructures support multicast natively, which can limit its applicability.

Broadcast Messaging

- Broadcast messaging involves sending a message from one sender to all nodes within the network. The key characteristics include:
 - ▣ **Wide Coverage:** The message is sent to every node, ensuring that all nodes in the network receive it.
 - ▣ **Network-Wide Reach:** Suitable for announcements, alerts, or updates intended for all nodes without targeting specific ones.
- **Advantages:**
 - ▣ Guarantees that every node in the network receives the message, which is useful for critical notifications or status updates.
 - ▣ Simplifies dissemination of information when all nodes need to be aware of an event or data change.
- **Disadvantages:**
 - ▣ Consumes significant network resources since every node, regardless of relevance, receives the message.
 - ▣ Can lead to unnecessary processing at nodes that don't need the message, potentially causing inefficiency.

Message-passing Programming Paradigm

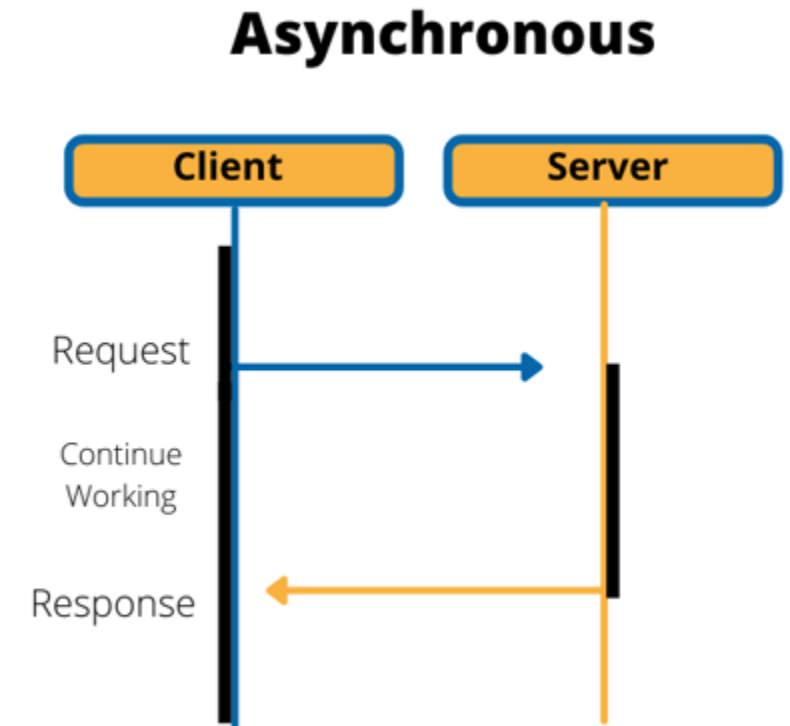
- Numerous programming languages (message passing paradigm) and libraries have been developed for ***explicit parallel programming***.
- The message-passing programming paradigm is one of the oldest and most widely used approaches for programming parallel computers.
- There are two key attributes that characterize the message-passing programming paradigm.
- The first is that it assumes a **partitioned address** space and the second is that it supports only **explicit parallelization**.
- The logical view of a machine supporting the message-passing paradigm consists of p processes, each with its own address space.
- Instances of such a view come naturally from clustered workstations and non-shared address space multicomputers.

Structure of Message-Passing Programs

- Message-passing programs are often written using the:
- **Asynchronous** paradigm.
- **Loosely synchronous** paradigm.

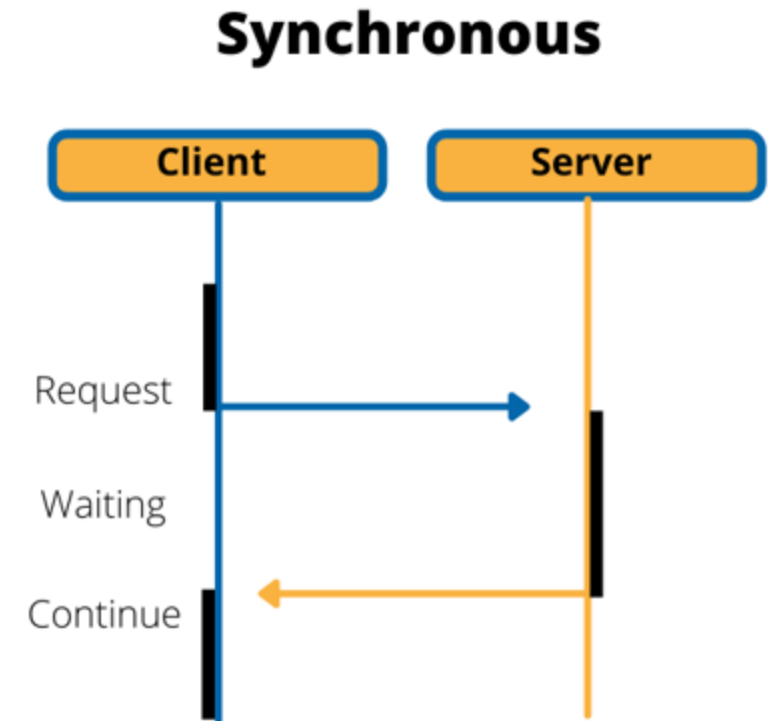
Asynchronous Paradigm

- In the **asynchronous paradigm**, all concurrent tasks execute asynchronously (no coordination).
- This makes it possible to implement any parallel algorithm.
- However, such programs can be harder to understand and can have ***nondeterministic behavior*** due to ***race conditions***.



Loosely Synchronous Paradigm

- **Loosely synchronous** programs are a *good compromise*.
- In such programs, tasks or subsets of tasks *synchronized to perform interactions*.
- However, between these interactions, *tasks execute completely asynchronously*.



The Building Blocks: Send and Receive Operations

- Since interactions are accomplished by ***sending and receiving messages***, the basic operations in the message-passing programming paradigm are send and receive.
- In their simplest form, the prototypes of these operations are defined as follows:
- ***send***(void *sendbuf, int nelems, int dest)
- ***receive***(void *recvbuf, int nelems, int source)
- The **sendbuf** points to a buffer that stores the data to be sent and **recvbuf** points to a buffer that stores the data to be **received**
- The **dest** is the identifier of the process that receives the data and **source** is the identifier of the process that sends the data

Send and Receive Operations

- The important thing to note is that process P0 changes the **value of a** to 0 immediately following the send.
- The semantics of the send operation require that the value received by process P1 must be 100 as opposed to 0.

P0

```
a = 100;  
send(&a, 1, 1);  
a=0;
```

P1

```
receive(&a, 1, 0)  
printf("%d\n", a);
```

Send and Receive Operations

- Most message passing platforms have additional hardware support for sending and receiving messages.
 1. They may support **DMA** (direct memory access) and
 2. Asynchronous message transfer using **network interface hardware**.

Send and Receive Operations

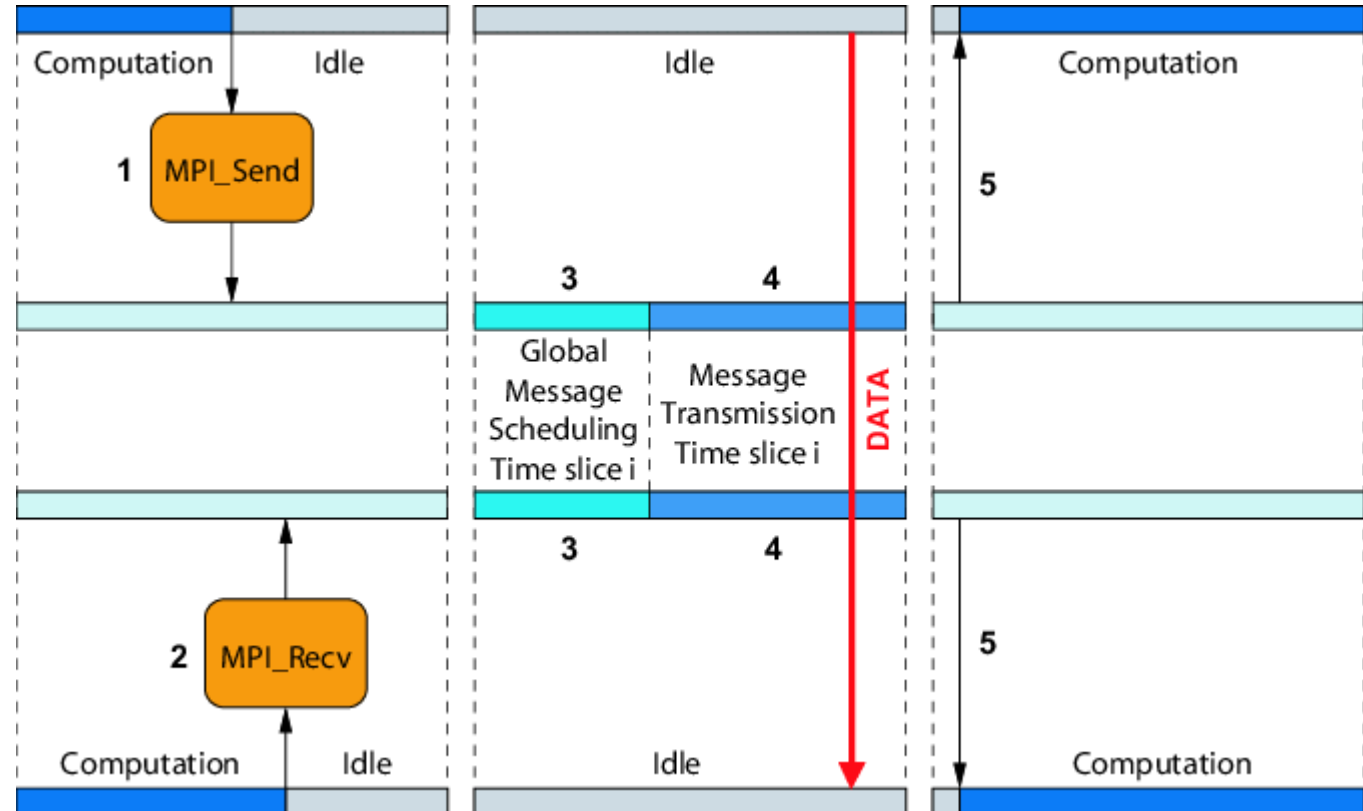
- Network interfaces allow the transfer of messages from buffer memory to desired location without CPU intervention.
- Similarly, DMA allows copying of data from one memory location to another (e.g., communication buffers) without CPU support (once they have been programmed).
- As a result, if the send operation programs the communication hardware and returns before the communication operation has been accomplished, process P1 might receive the value 0 instead of 100!

Blocking Message Passing Operations

- A simple solution to the problem presented in the code fragment above is for the send operation to return only when it is semantically safe to do so.
- It simply means that the sending operation blocks until it can guarantee that the semantics will not be violated on return irrespective of what happens in the program subsequently.
- There are two mechanisms by which this can be achieved:
 1. **Blocking Non-Buffered Send/Receive**
 2. **Blocking Buffered Send/Receive**

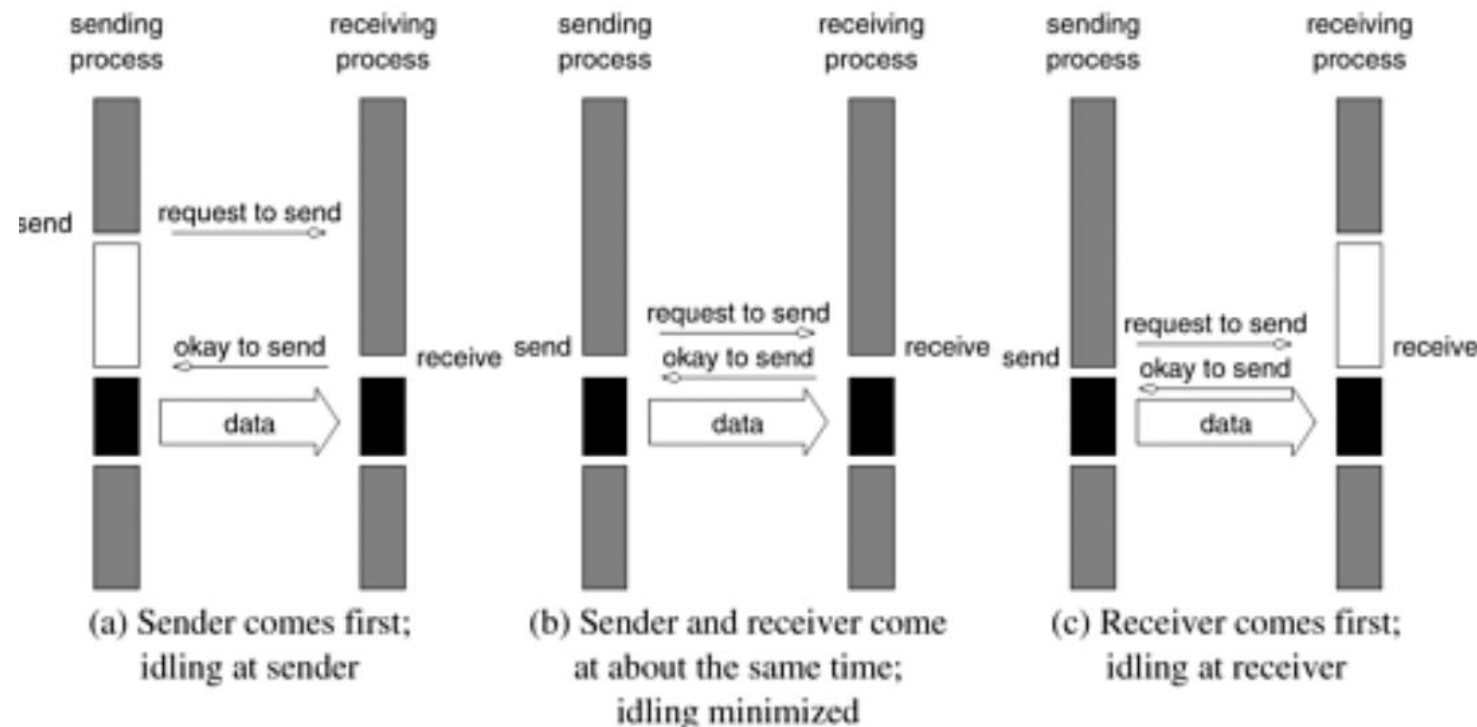
Blocking Non-Buffered Send/Receive:

- In the first case, the send operation does not return until the matching receive has been encountered at the receiving process.
- When this happens, the message is sent, and the send operation returns upon completion of the communication operation.



Blocking Message Passing Operations

- Typically, this process involves a handshake between the sending and receiving processes. The sending process sends a request to communicate to the receiving process.
- Since there are no buffers used at either sending or receiving ends, this is also referred to as a ***non-buffered blocking operation***.



Blocking Message Passing Operations

- In cases **(a)** and **(c)**, we notice that there is considerable **idling** at the sending and receiving process.
- It is also clear from the figures that a blocking non-buffered protocol is **suitable** when the send and receive are posted at roughly the **same time**.
- However, in an **asynchronous** environment, this may be **impossible** to predict. This **idling** overhead is one of the major **drawbacks** of this protocol.

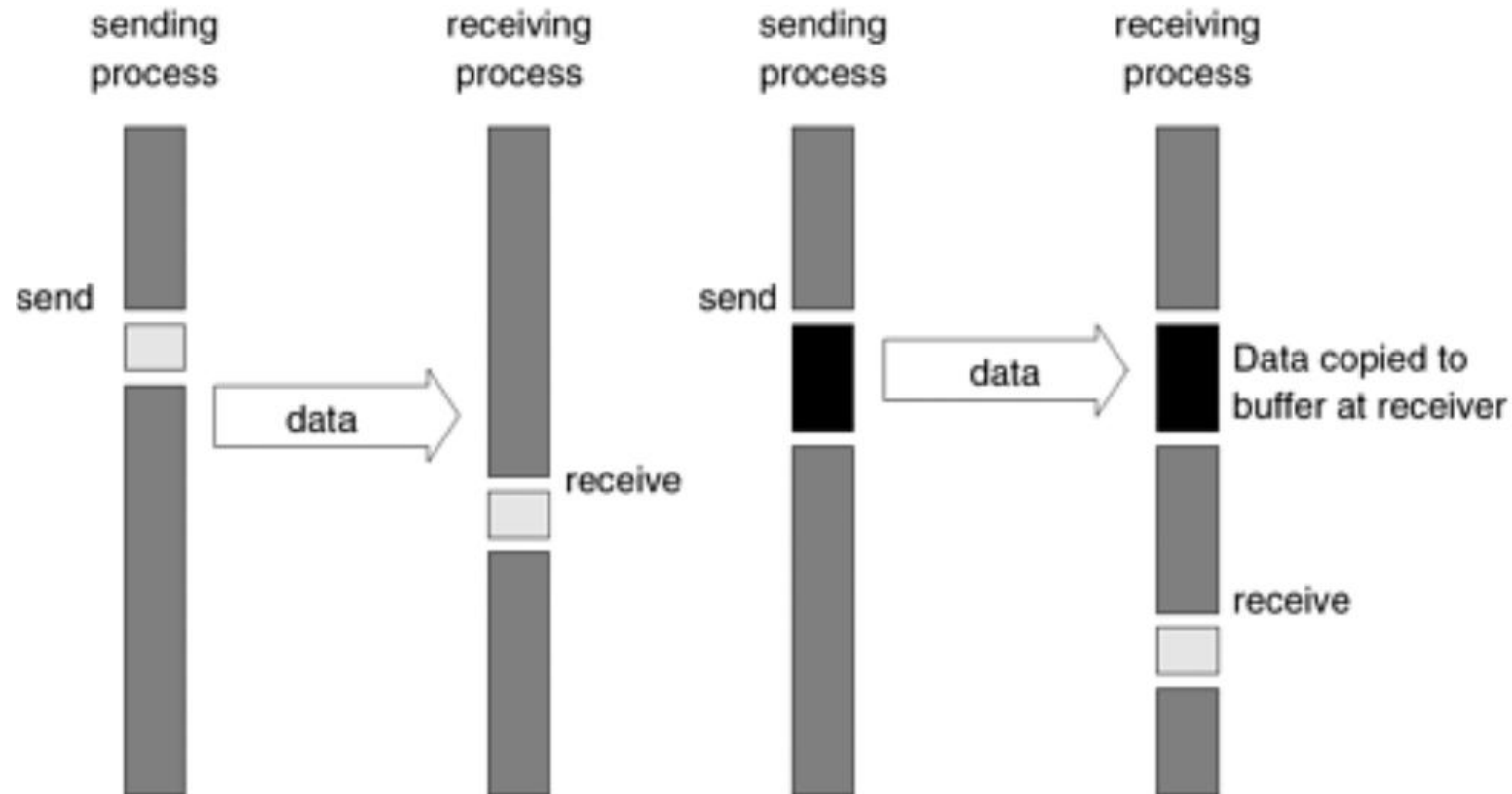
Blocking Buffered Send/Receive

- A simple solution to the idling and deadlocking problem outlined above is to rely on buffers at the sending and receiving ends.
- We start with a simple case in which the sender has a buffer pre-allocated for communicating messages.
- On encountering a send operation, the sender simply copies the data into the designated buffer and returns after the copy operation has been completed.

Blocking Message Passing Operations

- The sender process can now continue with the program knowing that any changes to the data will not impact program semantics.
- Note that at the receiving end, the data cannot be stored directly at the target location since this would violate program semantics.
- Instead, the data is copied into a buffer at the receiver as well.

Blocking Message Passing Operations



(a) presence of communication hardware with buffers at send and receive ends

(b) absence of communication hardware, sender interrupts receiver & deposits data in buffer at receiver end

Thanks

28



**"SOME BEAUTIFUL
PATHS CAN'T BE
DISCOVERED WITHOUT
GETTING LOST."**

EROL OZAN

hellofearless.com